Final Project Part 1

Introduction:

This project focuses on designing and implementing an ETL-based data integration system for an e-commerce company. The company collects customer feedback from multiple data sources, including CSV survey files, JSON web feedback forms, and XML external review platforms.

The purpose of this project is to integrate, clean, and analyze customer feedback data in order to improve product quality and customer satisfaction. The project includes database design, SQL query development, data integration techniques, and basic analytical reporting.

During this project, MySQL is used to create normalized relational database schemas, manage customer and product information, and analyze review data through SQL queries and data processing techniques. The final result helps identify top-performing products, customer engagement patterns, and recurring issues found in customer feedback.

Step 1: Understanding the Data Files

The project includes three different data sources used for customer feedback collection:

1. CSV File – Customer Survey Data

The CSV file contains structured customer survey information, including customer details, product information, ratings, comments, and review dates. This file represents internally collected survey responses.

Figure 1: Customer survey data stored in CSV format.

2. JSON File – Wb Feedback Forms

The JSON file contains customer feedback submitted through online web forms. The data is semi-structured and includes customer IDs, ratings, and customer comments.

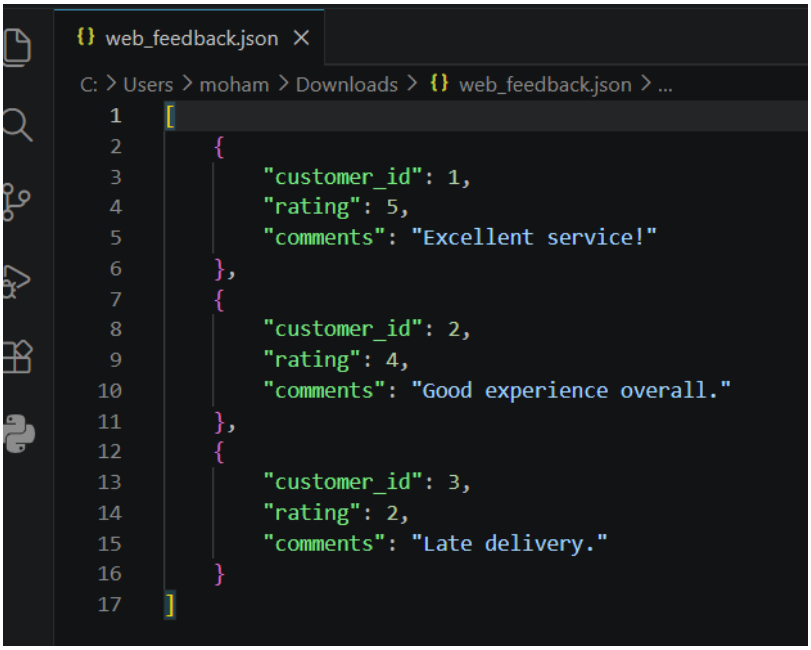


Figure 2: Customer feedback data collected from web forms in JSON format.

3. XML File – External Review Platform Data

The XML file contains external customer reviews collected from third-party review platforms. The data is stored in XML format and includes customer IDs, ratings, and comments.

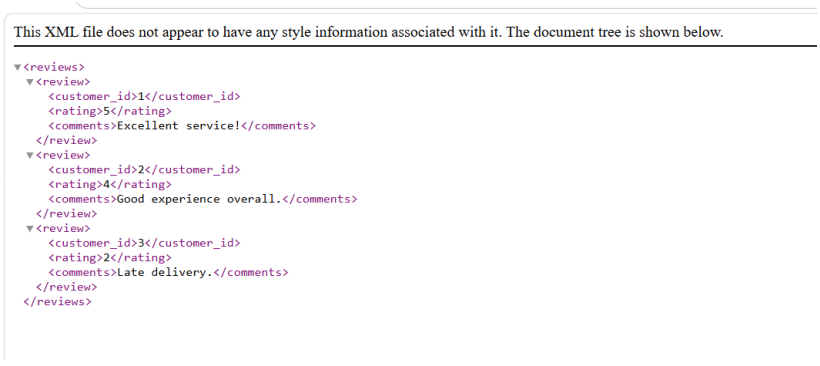


Figure 3: External customer reviews stored in XML format.

Step 2: Designing the ER Diagram

In this step, an Entity-Relationship Diagram (ERD) was designed to model the main entities in the e-commerce feedback system. The database includes Customers, Products, and Reviews. Each customer can write multiple reviews, and each product can receive multiple reviews. The Reviews table connects customers and products using foreign keys.

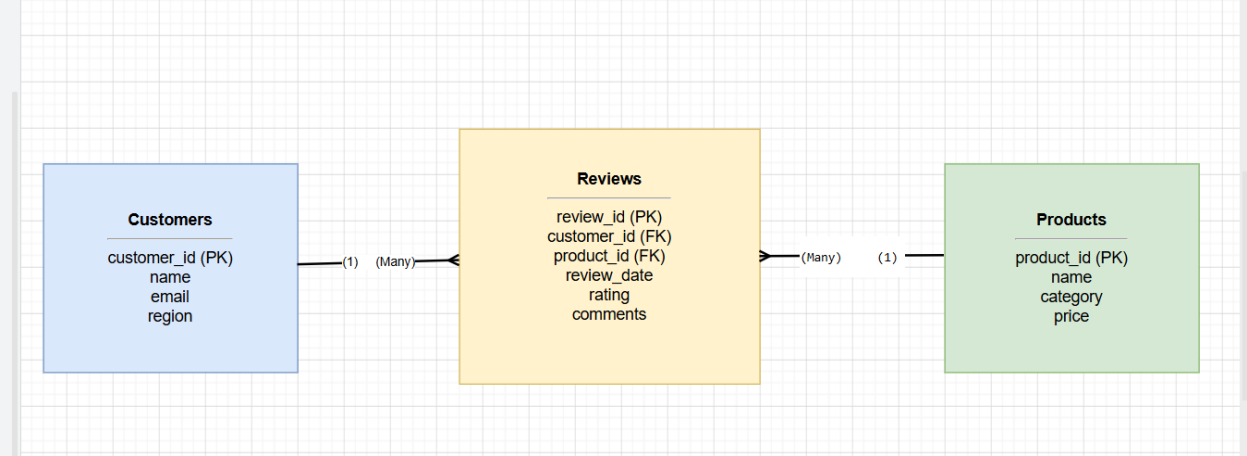
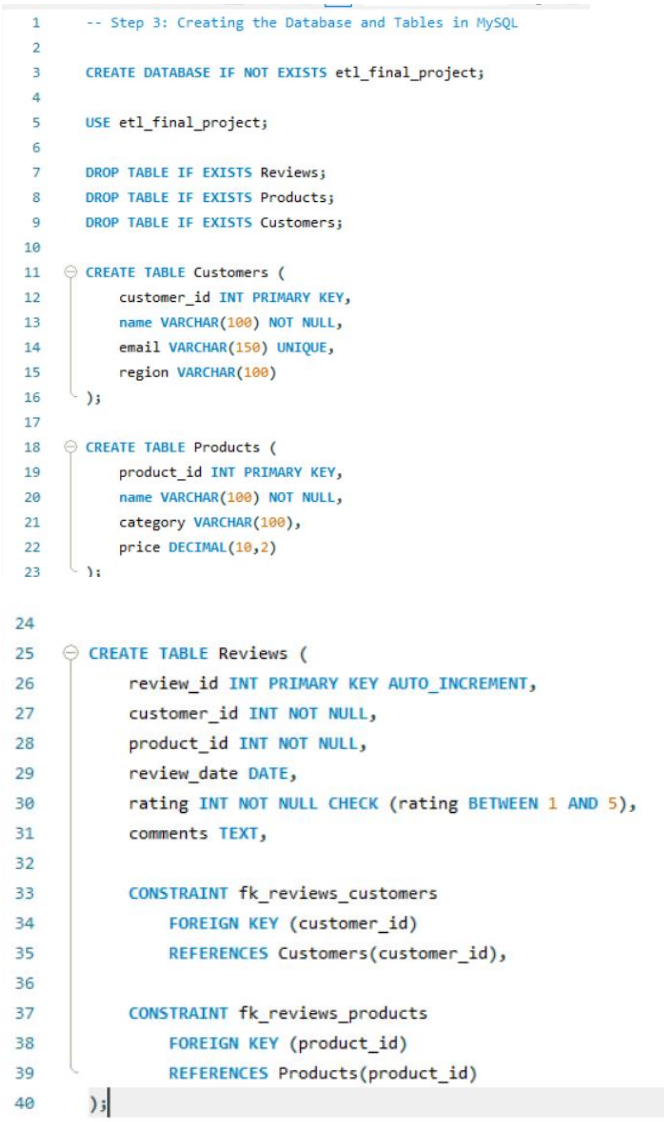


Figure 4: Entity-Relationship Diagram for the e-commerce customer feedback database.

Step 3: Creating the Database and Tables in MySQL

In this step, the relational database was created in MySQL. Three normalized tables were created: Customers, Products, and Reviews. Primary keys were used to uniquely identify records, and foreign keys were used to enforce relationships between customers, products, and reviews.



```sql
-- Step 3: Creating the Database and Tables in MySQL

CREATE DATABASE IF NOT EXISTS etl_final_project;

USE etl_final_project;

DROP TABLE IF EXISTS Reviews;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Customers;

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    region VARCHAR(100)
);

CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(100),
    price DECIMAL(10,2)
);


CREATE TABLE Reviews (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    review_date DATE,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comments TEXT,

    CONSTRAINT fk_reviews_customers
        FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id),

    CONSTRAINT fk_reviews_products
        FOREIGN KEY (product_id)
        REFERENCES Products(product_id)
);
```

Figure 5: Creating normalized database tables with primary keys and foreign keys in MySQL.

Step 4: Inserting Sample Data into the Tables

In this step, sample data was inserted into the Customers, Products, and Reviews tables. This data was used to test the database relationships and make sure that the primary keys and foreign keys work correctly before importing and integrating data from CSV, JSON, and XML files.

```sql
-- Step 4: Inserting Sample Data into the Tables

USE etl_final_project;

INSERT INTO Customers (customer_id, name, email, region)
VALUES
(1, 'John Doe', 'john.doe@example.com', 'North'),
(2, 'Jane Smith', 'jane.smith@example.com', 'East'),
(3, 'Emily Davis', 'emily.davis@example.com', 'South');

INSERT INTO Products (product_id, name, category, price)
VALUES
(101, 'Wireless Mouse', 'Electronics', 25.99),
(102, 'Bluetooth Headphones', 'Electronics', 59.99),
(103, 'Office Chair', 'Furniture', 149.99);

-- Add a general feedback product for reviews that do not include product_id
INSERT IGNORE INTO Products (product_id, name, category, price)
VALUES (999, 'General Customer Feedback', 'General Feedback', 0.00);

INSERT INTO Reviews (customer_id, product_id, review_date, rating, comments)
VALUES
(1, 101, '2024-01-01', 5, 'Great product!'),
(2, 102, '2024-01-02', 4, 'Good quality.'),
(3, 103, '2024-01-03', 3, 'Average quality.');
```

Figure 6: Inserting sample data into Customers, Products, and Reviews tables.

Queries are tested and work correctly:

Result Grid | Filter Rows: | Edit: | Export/Import:

review_id	customer_id	product_id	review_date	rating	comments
1	1	101	2024-01-01	5	Great product!
2	2	102	2024-01-02	4	Good quality.
3	3	103	2024-01-03	3	Average quality.
5	2	102	2024-01-02	4	Fast delivery.
NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/In

product_id	name	category	price
101	Wireless Mouse	Electronics	25.99
102	Bluetooth Headphones	Electronics	59.99
103	Office Chair	Furniture	149.99
999	General Customer Feedback	General Feedback	0.00
NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Ex

customer_id	name	email	region
1	John Doe	john.doe@example.com	North
2	Jane Smith	jane.smith@example.com	East
3	Emily Davis	emily.davis@example.com	South
NULL	NULL	NULL	NULL

Figure 7: Verifying inserted records in the Customers, Products, and Reviews tables.

Step 5: Importing CSV Data into MySQL

In this step, the customer_survey.csv file was imported into a staging table in MySQL. A staging table is used to temporarily store raw data before cleaning and moving it into the final relational tables.

```sql
-- Step 5: Importing CSV Data into MySQL

USE etl_final_project;

DROP TABLE IF EXISTS customer_survey_staging;

CREATE TABLE customer_survey_staging (
    customer_id INT,
    name VARCHAR(100),
    email VARCHAR(150),
    region VARCHAR(100),
    rating INT,
    comments TEXT,
    review_date DATE
);

DESCRIBE customer_survey_staging;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
customer_id	int	YES		NULL	
name	varchar(100)	YES		NULL	
email	varchar(150)	YES		NULL	
region	varchar(100)	YES		NULL	
rating	int	YES		NULL	
comments	text	YES		NULL	
review_date	date	YES		NULL	

Figure 8: Creating a staging table for importing CSV customer survey data.

```sql
-- Load CSV File into Staging Table

LOAD DATA LOCAL INFILE 'C:/Users/moham/Documents/Introduction to Extract-Transform-Load Assignments/Final Project Part 1/customer_sur
INTO TABLE customer_survey_staging
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

SELECT * FROM customer_survey_staging;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	email	region	rating	comments	review_date
1	John Doe	john.doe@example.com	North	5	Great product!	2024-01-01
2	Jane Smith	jane.smith@example.com	West	4	Fast delivery.	2024-01-02
3	Emily Davis	emily.davis@example.com	South	3	Average quality.	2024-01-03

Figure 9: Importing customer survey data from CSV into the staging table.

Note: The file path used in LOAD DATA LOCAL INFILE is based on my local computer. If the script is executed on another computer, the file path should be updated according to the location of the CSV file.

Step 6: Moving Cleaned CSV Data into Final Tables

In this step, cleaned customer and review records from the CSV staging table were moved into the final relational tables. Since the CSV file does not include product_id, the review records were assigned to a general feedback product using product_id 999.

Step 6 – Part 1: Moving Customer Data

```
121
122     -- Step 6 Part1: Moving CSV Data into Final Tables
123
124 •   INSERT IGNORE INTO Customers (customer_id, name, email, region)
125     SELECT
126         customer_id,
127         name,
128         email,
129         region
130     FROM customer_survey_staging;
131
132
133 •   SELECT * FROM Customers;
134
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	region
1	John Doe	john.doe@example.com	North
2	Jane Smith	jane.smith@example.com	East
3	Emily Davis	emily.davis@example.com	South
NULL	NULL	NULL	NULL

Step 6 – Part 2: Moving Review Data

```
135     -- Step 6 Part 2: Move CSV review data into the final Reviews table
136 •   INSERT INTO Reviews (customer_id, product_id, review_date, rating, comments)
137     SELECT
138         customer_id,
139         999 AS product_id,
140         review_date,
141         rating,
142         comments
143     FROM customer_survey_staging;
144
145
146
147 •   SELECT * FROM Reviews;
148
149
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

review_id	customer_id	product_id	review_date	rating	comments
16	1	999	2024-01-01	5	Great product!
17	2	999	2024-01-02	4	Fast delivery.
18	3	999	2024-01-03	3	Average quality.
NULL	NULL	NULL	NULL	NULL	NULL

Figure 10: Moving cleaned customer and review data from the CSV staging table into the final Customers and Reviews tables.

Step 6 – Part 3: Data Cleaning Using SQL UPDATE

```sql
-- Step 6 Part 3: Data Cleaning Using SQL UPDATE

SET SQL_SAFE_UPDATES = 0;

-- Remove extra spaces from review comments
UPDATE Reviews
SET comments = TRIM(comments);

-- Replace missing review dates with the current date
UPDATE Reviews
SET review_date = CURDATE()
WHERE review_date IS NULL;

SET SQL_SAFE_UPDATES = 1;

SELECT * FROM Reviews;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

review_id	customer_id	product_id	review_date	rating	comments
16	1	999	2024-01-01	5	Great product!
17	2	999	2024-01-02	4	Fast delivery.
18	3	999	2024-01-03	3	Average quality.
NULL	NULL	NULL	NULL	NULL	NULL

Step 7: Parsing JSON Data

The JSON file was successfully opened and parsed using Python's json library. The extracted data was stored in a Python variable for further processing and integration. The parsed JSON feedback data was converted into a structured pandas DataFrame to support integration and analysis within the ETL workflow.

Figure 11: Reading and parsing the web_feedback.json file using Python.

Each feedback record from the JSON file was displayed individually using a Python loop. This helped verify that the JSON data was parsed correctly and converted into a structured format.

Figure 12: Displaying individual feedback records parsed from the JSON file.

The parsed JSON data was converted into a pandas DataFrame to organize the feedback records into a tabular structure suitable for data analysis and integration.

Figure 13: Converting parsed JSON data into a pandas DataFrame.

The DataFrame structure and data types were verified using the info() function. This step confirmed that the JSON data was successfully converted into a structured format with valid records and appropriate data types.

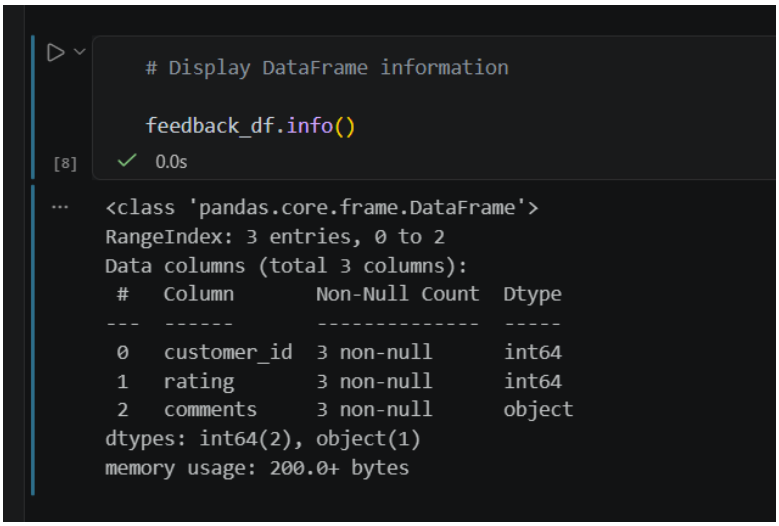


Figure 14: Verifying the structure and data types of the parsed JSON DataFrame.

Step 8: Parsing XML Data

In this step, the external_reviews.xml file was parsed using Python's ElementTree library. The XML data was extracted and converted into a structured format to support data integration and analysis within the ETL process. The root element of the XML structure was identified to verify that the file was read correctly. The parsed XML review data was transformed into a structured pandas DataFrame for easier integration, validation, and analysis in the ETL workflow.

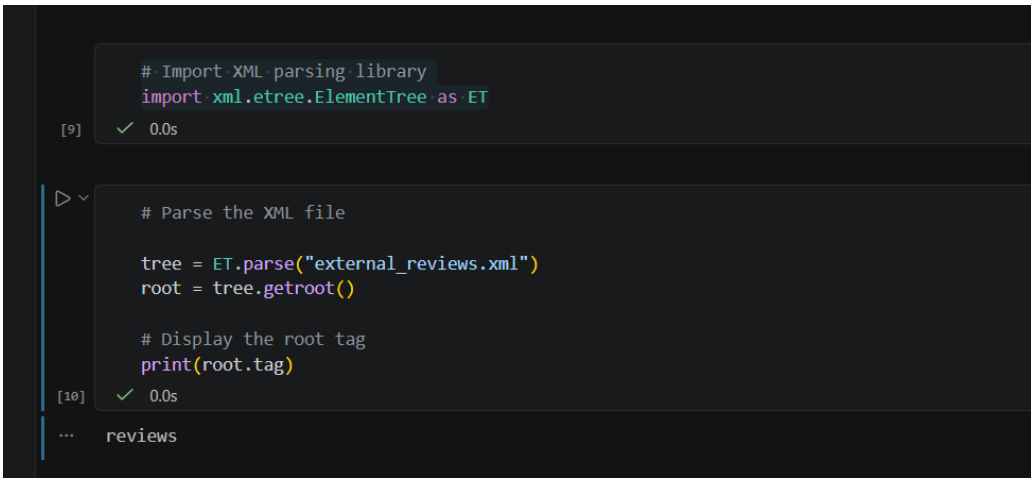


Figure 15: Parsing the external_reviews.xml file and displaying the root XML tag

The review records were extracted from the XML file by iterating through each review element and retrieving the customer ID, rating, and comments fields.

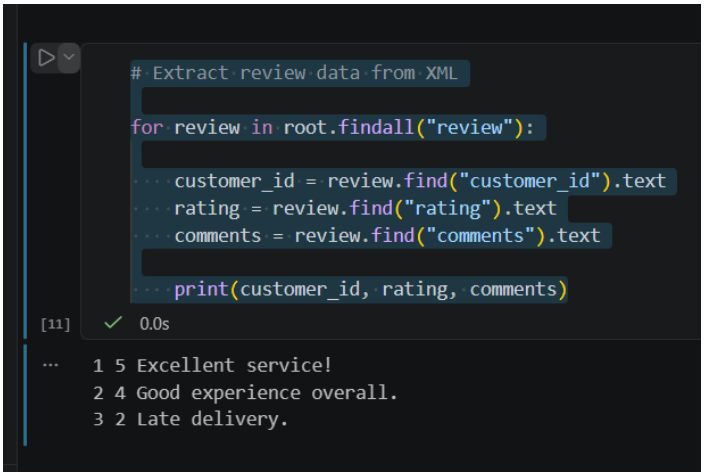


Figure 16: Extracting customer review records from the XML file.

The extracted XML review data was stored in a Python list and converted into a pandas DataFrame. This transformation organized the XML records into a structured tabular format suitable for ETL processing and analysis.

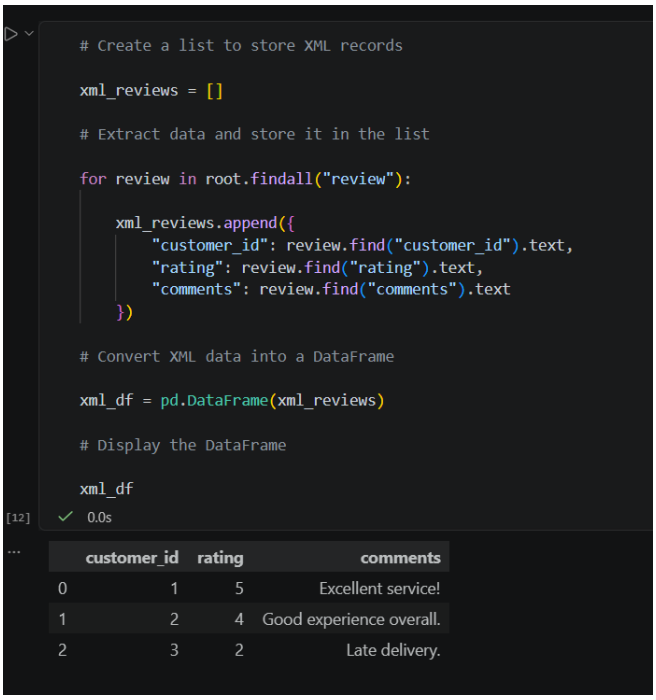


Figure 17: Converting extracted XML review data into a pandas DataFrame.

The structure and data types of the XML DataFrame were verified using the info() function. This validation step confirmed that the XML data was successfully transformed into a structured format with complete records.

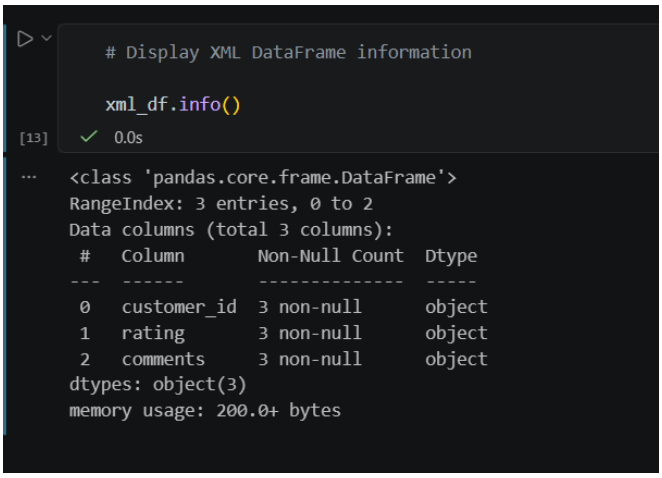


Figure 18: Verifying the structure and data types of the XML DataFrame.

Step 9: Creating SQL Views and Joins

In this step, SQL joins were used to combine customer, product, and review data into one integrated view. This view makes it easier to analyze customer feedback, product performance, and review information in a structured way.

The customer_product_reviews view was created using LEFT JOIN operations to combine customer, product, and review information into a single integrated dataset.

```sql
-- Step 9: Creating SQL Views and Joins

CREATE OR REPLACE VIEW customer_product_reviews AS
SELECT
    r.review_id,
    c.customer_id,
    c.name AS customer_name,
    c.email,
    c.region,
    p.product_id,
    p.name AS product_name,
    p.category,
    p.price,
    r.review_date,
    r.rating,
    r.comments
FROM Reviews r
LEFT JOIN Customers c
    ON r.customer_id = c.customer_id
LEFT JOIN Products p
    ON r.product_id = p.product_id;



SELECT * FROM customer_product_reviews;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

review_id	customer_id	customer_name	email	region	product_id	product_name	category	price	review_date	rating	comments
1	1	John Doe	john.doe@example.com	North	101	Wireless Mouse	Electronics	25.99	2024-01-01	5	Great product!
2	2	Jane Smith	jane.smith@example.com	East	102	Bluetooth Headphones	Electronics	59.99	2024-01-02	4	Good quality.
3	3	Emily Davis	emily.davis@example.com	South	103	Office Chair	Furniture	149.99	2024-01-03	3	Average quality
5	2	Jane Smith	jane.smith@example.com	East	102	Bluetooth Headphones	Electronics	59.99	2024-01-02	4	Fast delivery.

Figure 19: Creating and displaying the customer_product_reviews SQL view using LEFT JOIN operations.

Step 10: SQL Analysis Queries

In this step, SQL analysis queries were used to evaluate product performance and customer feedback. Aggregate functions such as AVG() and COUNT() were applied to calculate average ratings and review counts for each product. The query analyzed product performance based on customer reviews using AVG(), COUNT(), GROUP BY, and ORDER BY SQL clauses.

```
171
172     -- Step 10: SQL Analysis Queries
173
174 •   SELECT
175         p.product_id,
176         p.name AS product_name,
177         AVG(r.rating) AS average_rating,
178         COUNT(r.review_id) AS total_reviews
179     FROM Products p
180     LEFT JOIN Reviews r
181         ON p.product_id = r.product_id
182     GROUP BY
183         p.product_id,
184         p.name
185     ORDER BY
186         average_rating DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	average_rating	total_reviews
101	Wireless Mouse	5.0000	1
102	Bluetooth Headphones	4.0000	2
103	Office Chair	3.0000	1

Figure 20: SQL analysis query showing average product ratings and total review counts.

Step 11: Creating an Index for Query Optimization

In this step, SQL indexes were created on the product_id, customer_id, and review_date columns in the Reviews table to improve query performance. These indexes help speed up JOIN operations, filtering, and date-based analysis. The SHOW INDEX statement was used to verify that the indexes were successfully created.

```
220
221     -- Step 11: Creating an Index for Query Optimization
222
223 •   CREATE INDEX idx_reviews_product
224     ON Reviews(product_id);
225
226
227 •   CREATE INDEX idx_reviews_customer
228     ON Reviews(customer_id);
229
230 •   CREATE INDEX idx_reviews_date
231     ON Reviews(review_date);
232
233 •   SHOW INDEX FROM Reviews;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
reviews	0	PRIMARY	1	review_id	A	3	NULL	NULL		BTREE			YES	NULL
reviews	1	idx_reviews_product	1	product_id	A	3	NULL	NULL		BTREE			YES	NULL
reviews	1	idx_reviews_customer	1	customer_id	A	3	NULL	NULL		BTREE			YES	NULL
reviews	1	idx_reviews_date	1	review_date	A	3	NULL	NULL	YES	BTREE			YES	NULL

Figure 21: Creating and verifying an SQL index on the Reviews table for query optimization.

Conclusion and Findings:

In this project, data from multiple sources including CSV, JSON, and XML files was successfully integrated into a relational MySQL database. ETL techniques were applied to import, parse, clean, and organize customer feedback and product review data. SQL joins, views, aggregate queries, and indexing were used to analyze product performance and optimize query execution. The project demonstrated how structured and semi-structured data can be transformed into meaningful analytical information to support business decision-making and customer satisfaction analysis.